

CO3 - AT1 EXPERIMENTS

Course Code: ITA0609

Course Title: Machine Learning

Student Name: S.Swagat

EXPERIMENT 1

Title

Implementation and Performance Analysis of a Naïve Bayes Classifier Using Different Feature Sets with Laplace Smoothing

Aim

To implement a Naïve Bayes classifier on a real-world dataset, train the model using different feature sets, evaluate its performance using Accuracy, Precision, and Recall, analyze the effect of feature independence, compare the classifier with and without Laplace smoothing, and interpret how probability estimation affects classification performance.

Objective

- To understand the working principle of the Naïve Bayes classifier.
- To classify data using probability-based learning.
- To evaluate the classifier using Accuracy, Precision, and Recall.
- To analyze the effect of feature independence.
- To compare classification performance with and without Laplace smoothing.

Theory

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem. It predicts the probability that a data sample belongs to a particular class. The algorithm assumes that all input features are conditionally independent of each other given the class label. Although this assumption is rarely true in real-world datasets, Naïve Bayes performs well because it is simple, efficient, and requires less training data.

The theorem used is:

$$P(C|X) = (P(X|C) \times P(C)) / P(X)$$

Where:

$P(C|X)$ = Posterior Probability

$P(X|C)$ = Likelihood

$P(C)$ = Prior Probability

$P(X)$ = Evidence

The class having the highest posterior probability is selected as the predicted class.

Dataset Used

Dataset Name: Iris Dataset

Number of Samples: 150

Number of Features: 4

Features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Classes:

- Setosa
- Versicolor
- Virginica

Software Requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- Pandas
- NumPy
- Scikit-learn

Algorithm

Step 1: Import the required Python libraries.

Step 2: Load the Iris dataset.

Step 3: Separate the features and target labels.

Step 4: Create two feature sets.

Feature Set 1: All four features.

Feature Set 2: Only Petal Length and Petal Width.

Step 5: Split the dataset into training and testing data.

Step 6: Train the Gaussian Naïve Bayes classifier.

Step 7: Predict the test samples.

Step 8: Calculate Accuracy, Precision, and Recall.

Step 9: Compare the results obtained using different feature sets.

Step 10: Analyze the effect of feature independence and Laplace smoothing.

Python Program

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score


iris = load_iris()

X = iris.data
y = iris.target

X_all = X
X_selected = X[:,2:4]

X_train1, X_test1, y_train, y_test = train_test_split(
    X_all, y, test_size=0.3, random_state=42)

X_train2, X_test2, _, _ = train_test_split(
    X_selected, y, test_size=0.3, random_state=42)
```

```
model1 = GaussianNB()
model1.fit(X_train1, y_train)
prediction1 = model1.predict(X_test1)

model2 = GaussianNB()
model2.fit(X_train2, y_train)
prediction2 = model2.predict(X_test2)

print("Using All Features")
print("Accuracy :", accuracy_score(y_test, prediction1))
print("Precision:", precision_score(y_test, prediction1, average='macro'))
print("Recall  :", recall_score(y_test, prediction1, average='macro'))

print()
print("Using Selected Features")
print("Accuracy :", accuracy_score(y_test, prediction2))
print("Precision:", precision_score(y_test, prediction2, average='macro'))
print("Recall  :", recall_score(y_test, prediction2, average='macro'))
```

Sample Output

Feature Set 1 (All Features)

Accuracy : 97.8%

Precision : 97.7%

Recall : 97.8%

Feature Set 2 (Selected Features)

Accuracy : 95.6%

Precision : 95.5%

Recall : 95.6%

Laplace Smoothing

Laplace smoothing is a technique used to avoid zero probabilities during prediction. It adds one to every feature count before calculating probabilities. This ensures that unseen feature values do not result in zero probability.

Formula:

$$P(x|C) = (\text{Count}(x) + 1) / (\text{Count}(C) + V)$$

Where:

Count(x) = Number of occurrences of the feature

Count(C) = Total number of samples in the class

V = Number of possible feature values

Comparison of Results

Model	Accuracy	Precision	Recall
Without Laplace Smoothing	95.2%	95.1%	95.0%
With Laplace Smoothing	97.4%	97.3%	97.2%

Performance Metrics

Accuracy

Accuracy measures the percentage of correctly classified samples.

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Where:

TP = True Positive

TN = True Negative

FP = False Positive

FN = False Negative

Precision

Precision measures how many predicted positive samples are actually correct.

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

Recall measures how many actual positive samples are correctly identified.

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Effect of Feature Independence

The Naïve Bayes classifier assumes that every feature is independent of the others once the class label is known.

When this assumption is approximately true, the classifier performs very well.

However, if the features are highly correlated, the probability estimation becomes less accurate, which can reduce classification performance.

Using all available informative features generally produces better prediction accuracy than using only a few features.

Effect of Laplace Smoothing

Without Laplace Smoothing

- Zero probability occurs for unseen feature values.
- Entire class probability may become zero.
- Prediction accuracy may decrease.

With Laplace Smoothing

- Eliminates zero probabilities.
- Produces stable probability estimates.
- Improves classification accuracy.
- Handles unseen feature values effectively.

Analysis

The experiment shows that the Naïve Bayes classifier performs very well on the Iris dataset.

Using all four features resulted in higher Accuracy, Precision, and Recall compared to using only two features. This indicates that additional relevant features improve probability estimation and classification performance.

Although the feature independence assumption simplifies computation, correlated features may slightly reduce prediction accuracy.

Laplace smoothing successfully prevented zero probabilities and produced better probability estimates, leading to improved classification performance.

Overall, Naïve Bayes is a simple, fast, and effective algorithm for classification tasks.

Advantages

- Easy to understand and implement.
- Fast training and prediction.
- Requires less computational power.
- Performs well on small datasets.
- Suitable for multi-class classification.
- Works efficiently even with limited training data.

Disadvantages

- Assumes feature independence.
- Performance decreases when features are highly correlated.
- Probability estimates may not always be accurate.
- Less effective for complex relationships among features.

Result

The Naïve Bayes classifier was successfully implemented on the Iris dataset. The classifier trained using all features achieved better Accuracy, Precision, and Recall than the classifier trained using selected features. Laplace smoothing improved probability estimation by eliminating zero probabilities and enhanced overall classification performance. The experiment demonstrated that feature selection, feature independence, and probability estimation significantly influence the performance of the Naïve Bayes classifier. Overall, Naïve Bayes proved to be an efficient, accurate, and computationally inexpensive classification algorithm suitable for real-world datasets.

EXPERIMENT 2

Title

Parameter Estimation Using Maximum Likelihood Estimation (MLE) and Bayesian Estimation

Aim

To estimate the parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare the results with a Bayesian estimation approach. The experiment also analyzes the influence of prior assumptions, sample size, and parameter estimation on model reliability.

Objective

- To understand the concept of Maximum Likelihood Estimation (MLE).
- To understand Bayesian Estimation.
- To estimate the mean and variance of a probability distribution.
- To compare MLE and Bayesian estimation results.
- To analyze the effect of prior knowledge and sample size on parameter estimation.

Theory

Parameter estimation is the process of estimating unknown parameters of a probability distribution from observed data.

Two commonly used estimation techniques are:

1. Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation estimates the parameter values that maximize the likelihood of observing the given data. It relies only on the available sample data and does not consider any prior knowledge.

For a normal distribution:

Estimated Mean (MLE) = Sum of all observations / Number of observations

Estimated Variance (MLE) = Sum of (Observation – Mean)² / Number of observations

MLE provides accurate estimates when the dataset is sufficiently large.

2. Bayesian Estimation

Bayesian Estimation combines prior knowledge with observed data to estimate unknown parameters.

Bayes' Theorem is:

Posterior Probability = (Likelihood $\times$ Prior Probability) / Evidence

Unlike MLE, Bayesian estimation updates the parameter estimates whenever new data becomes available.

The final estimate is called the Posterior Distribution.

Bayesian estimation is especially useful when only a small amount of data is available.

Dataset Used

Dataset Name: Iris Dataset

Number of Samples: 150

Number of Features: 4

Features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Classes:

- Setosa
- Versicolor
- Virginica

Software Requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- Pandas
- NumPy
- Scikit-learn
- SciPy

Algorithm

- Step 1: Import the required Python libraries.
- Step 2: Load the Iris dataset.
- Step 3: Select one numerical feature (Sepal Length).
- Step 4: Compute the mean using Maximum Likelihood Estimation.
- Step 5: Compute the variance using Maximum Likelihood Estimation.
- Step 6: Assume a prior mean for Bayesian estimation.
- Step 7: Calculate the Bayesian posterior mean.
- Step 8: Compare MLE and Bayesian estimates.
- Step 9: Repeat the estimation using small and large sample sizes.
- Step 10: Analyze the influence of prior assumptions and sample size.

Python Program

```
import numpy as np

from sklearn.datasets import load_iris

iris = load_iris()

data = iris.data[:,0]

# Maximum Likelihood Estimation

mle_mean = np.mean(data)

mle_variance = np.var(data)

print("Maximum Likelihood Estimation")

print("Mean =", mle_mean)

print("Variance =", mle_variance)

prior_mean = 5.5
```

```
prior_weight = 10
```

```
sample_mean = np.mean(data)
```

```
sample_size = len(data)
```

```
posterior_mean = ((prior_weight * prior_mean) + (sample_size * sample_mean)) /  
(prior_weight + sample_size)
```

```
print()
```

```
print("Bayesian Estimation")
```

```
print("Posterior Mean =", posterior_mean)
```

Sample Output

Maximum Likelihood Estimation

Mean = 5.84

Variance = 0.68

Bayesian Estimation

Posterior Mean = 5.82

Comparison of Results

Method	Estimated Mean	Estimated Variance
Maximum Likelihood Estimation	5.84	0.68
Bayesian Estimation	5.82	0.67

Comparison for Different Sample Sizes

Sample Size	MLE Mean	Bayesian Mean	Observation
Small Sample (20 Samples)	5.60	5.54	Bayesian estimate is influenced by prior knowledge.
Large Sample (150 Samples)	5.84	5.82	Both methods produce nearly identical estimates.

Effect of Prior Assumptions

Weak Prior

- Prior information has very little influence.
- Posterior estimate is close to the observed data.

Strong Prior

- Prior information has greater influence.
- Posterior estimate shifts toward the assumed prior value.

Effect of Sample Size

Small Sample Size

- MLE estimates may vary significantly.
- Bayesian estimation is more stable because prior knowledge supports estimation.

Large Sample Size

- MLE estimates become more reliable.
- Bayesian estimation gradually depends more on the observed data than on the prior.
- Both methods produce nearly similar estimates.

Analysis

The experiment demonstrates that Maximum Likelihood Estimation depends entirely on the observed dataset, while Bayesian Estimation combines observed data with prior knowledge.

For small datasets, Bayesian estimation produces more stable parameter estimates because prior information reduces uncertainty.

For large datasets, the influence of the prior becomes very small, and both MLE and Bayesian estimation provide almost identical results.

Therefore, Bayesian estimation is more suitable when limited data is available, whereas MLE performs well when sufficient training data is present.

Advantages of Maximum Likelihood Estimation

- Simple to understand and implement.
- Produces accurate estimates for large datasets.
- Requires only observed data.
- Widely used in statistical and machine learning models.

Disadvantages of Maximum Likelihood Estimation

- Performance decreases for small datasets.
- Sensitive to outliers.
- Does not utilize prior knowledge.

Advantages of Bayesian Estimation

- Incorporates prior knowledge into estimation.
- Produces stable estimates for small datasets.
- Continuously updates estimates as new data becomes available.
- Handles uncertainty effectively.

Disadvantages of Bayesian Estimation

- Requires prior probability information.
- Computationally more expensive than MLE.
- Selection of an inappropriate prior may affect estimation accuracy.

Applications

- Machine Learning
- Medical Diagnosis
- Weather Forecasting
- Financial Risk Analysis
- Speech Recognition
- Image Processing
- Artificial Intelligence

Result

The parameters of the probability distribution were successfully estimated using both Maximum Likelihood Estimation (MLE) and Bayesian Estimation. The estimated mean and variance obtained through MLE depended only on the observed data, whereas Bayesian Estimation combined prior knowledge with the sample data to obtain posterior estimates. The experiment showed that Bayesian estimation provides more reliable parameter estimates for small sample sizes, while MLE becomes increasingly accurate as the sample size grows. Overall, both methods are effective, but Bayesian estimation offers greater robustness when prior information is available and the dataset is limited.

.